
Application of Iterative and Decision-Making Constructs to Shapes and Physics using Modular Programming in C

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16
Date	13 September 2026

1. Aim

To prepare an algorithm and a test-case table for each operation, and to develop, compile, and execute a menu-driven modular C project that calculates the geometrical measurements and physical properties (surface area, volume, base area, mass, weight, gravitational potential energy, pressure, buoyant force, and buoyancy behavior) of a sphere, cylinder, cone, and cuboid.

2. Constraints Followed

- Only fundamental data types (int, double) are used.
- All functions use call by value; no pointer variables or pointer parameters are declared.
- No recursion, arrays, structures, unions, enumerations, or typedef is used.
- No function from math.h is used — square root is computed iteratively (Newton–Raphson) in math_utils.c.
- Only selection statements (if-else, switch-case) and loops (do-while) are used for control flow.

3. Project Organisation

File	Responsibility
solid.h	Header guard, symbolic constants, and external declarations for all functions.
math_utils.c	Iterative (Newton–Raphson) square root — calculate_square_root().
geometry.c	Surface area, volume, base area, and slant height (via math_utils.c).
physics.c	Obtains solid type/dimensions for physics operations and performs the calculations.
main.c	Displays the menu, obtains common inputs, calls the required functions, displays results.

4. Algorithms

4.1 math_utils.c

calculate_square_root() double calculate_square_root(double number)

1. Start.
2. Receive number.
3. If number < 0, return -1 (no real square root) and stop.
4. If number == 0, return 0 and stop.
5. Set old_value = number.
6. Repeat (do-while):
 7. a. new_value = 0.5 * (old_value + number / old_value).
 8. b. difference = new_value - old_value.
 9. c. If difference < 0, make difference positive.
 10. d. old_value = new_value.
11. ...while difference >= TOLERANCE.
12. Return new_value.
13. Stop.

4.2 geometry.c

sphere_area() double sphere_area(double radius)

1. Start. Receive radius.
2. $\text{area} = 4 * \text{PI} * \text{radius} * \text{radius}$.
3. Return area. Stop.

sphere_volume() double sphere_volume(double radius)

1. Start. Receive radius.
2. $\text{volume} = (4/3) * \text{PI} * \text{radius}^3$.
3. Return volume. Stop.

cylinder_area() double cylinder_area(double radius, double height)

1. Start. Receive radius, height.
2. $\text{area} = 2 * \text{PI} * \text{radius} * (\text{radius} + \text{height})$.
3. Return area. Stop.

cylinder_volume() double cylinder_volume(double radius, double height)

1. Start. Receive radius, height.
2. $\text{volume} = \text{PI} * \text{radius} * \text{radius} * \text{height}$.
3. Return volume. Stop.

cone_area() double cone_area(double radius, double height)

1. Start. Receive radius, height.
2. $\text{slant_height} = \text{calculate_square_root}(\text{radius} * \text{radius} + \text{height} * \text{height})$.
3. $\text{area} = \text{PI} * \text{radius} * (\text{radius} + \text{slant_height})$.
4. Return area. Stop.

cone_volume() double cone_volume(double radius, double height)

1. Start. Receive radius, height.
2. $\text{volume} = (1/3) * \text{PI} * \text{radius} * \text{radius} * \text{height}$.
3. Return volume. Stop.

cuboid_area() double cuboid_area(double length, double breadth, double height)

1. Start. Receive length, breadth, height.
2. $\text{area} = 2 * (\text{length} * \text{breadth} + \text{breadth} * \text{height} + \text{length} * \text{height})$.
3. Return area. Stop.

cuboid_volume() double cuboid_volume(double length, double breadth, double height)

1. Start. Receive length, breadth, height.
2. $\text{volume} = \text{length} * \text{breadth} * \text{height}$.
3. Return volume. Stop.

circular_base_area() double circular_base_area(double radius)

1. Start. Receive radius.
2. $\text{base_area} = \text{PI} * \text{radius} * \text{radius}$.
3. Return base_area. Stop.

cuboid_base_area() double cuboid_base_area(double length, double breadth)

1. Start. Receive length, breadth.
2. $\text{base_area} = \text{length} * \text{breadth}$.
3. Return base_area. Stop.

4.3 physics.c

calculate_mass() double calculate_mass(double density)

1. Start. Receive density.
2. Obtain and validate the solid type (1–4).
3. Obtain and validate the dimensions required for that solid.
4. Compute volume using the matching geometry.c function.
5. mass = density * volume.
6. Return mass. Stop.

calculate_weight() double calculate_weight(void)

1. Start.
2. Obtain and validate density (> 0).
3. mass = calculate_mass(density).
4. weight = mass * GRAVITY.
5. Return weight. Stop.

potential_energy() double potential_energy(double elevation)

1. Start. Receive elevation.
2. Obtain and validate density (> 0).
3. mass = calculate_mass(density).
4. energy = mass * GRAVITY * elevation.
5. Return energy. Stop.

calculate_pressure() double calculate_pressure(double density)

1. Start. Receive density.
2. Obtain and validate the solid type, restricted to cylinder / cone / cuboid.
3. Obtain and validate the dimensions required for that solid.
4. contact_area = circular_base_area() for cylinder/cone, or cuboid_base_area() for cuboid.
5. volume = the matching volume function.
6. pressure = (density * volume * GRAVITY) / contact_area.
7. Return pressure. Stop.

buoyant_force() double buoyant_force(double fluid_density)

1. Start. Receive fluid_density.
2. Obtain and validate the solid type (1–4).
3. Obtain and validate the dimensions required for that solid.
4. volume = the matching volume function.
5. force = fluid_density * volume * GRAVITY.
6. Return force. Stop.

determine_buoyancy() int determine_buoyancy(double solid_density, double fluid_density)

1. Start. Receive solid_density, fluid_density.
2. If solid_density > fluid_density, return SINKS.
3. Else if solid_density < fluid_density, return FLOATS.
4. Else, return SUSPENDED.
5. Stop.

4.4 main()

main() int main(void)

1. Start.
2. Repeat (do-while) until choice == 10:
3. a. Display the menu (options 1–10) and read choice.
4. b. If choice is 1, 2, or 3 (Surface Area / Volume / Base Area): display the solid submenu, obtain and validate the solid choice and its dimensions in main(), call the matching geometry.c function, and display the result. For Base Area, if Sphere is chosen, display a message and return to the menu instead of computing.
5. c. If choice = 4 (Mass): obtain and validate density; call calculate_mass(density); display the mass.
6. d. If choice = 5 (Weight): call calculate_weight(); display the weight.
7. e. If choice = 6 (Potential Energy): obtain and validate elevation (≥ 0); call potential_energy(elevation); display the energy.
8. f. If choice = 7 (Pressure): obtain and validate density; call calculate_pressure(density); display the pressure.
9. g. If choice = 8 (Buoyant Force): obtain and validate fluid density; call buoyant_force(fluid_density); display the force.
10. h. If choice = 9 (Buoyancy Behavior): obtain and validate solid density and fluid density; call determine_buoyancy(); display Sinks / Floats / Suspended accordingly.
11. i. If choice = 10, display an exit message.
12. j. Otherwise, display an invalid-choice message.
13. Stop when choice == 10.

5. Test-Case Tables

Each operation was tested with valid inputs, invalid inputs (to verify the immediate re-validation loop), and boundary inputs. All actual outputs matched the expected (hand-calculated) values; execution transcripts are in Section 7.

5.1 Solid Choice & Dimension Validation

#	Test Case	Input	Expected Result	Actual Result
1	Solid choice below range (invalid)	solid choice = 0	Error message; re-prompt	Pass
2	Solid choice above range (invalid)	solid choice = 5	Error message; re-prompt	Pass
3	Solid choice valid	solid choice = 1 (Sphere)	Accepted, proceeds	Pass
4	Dimension negative (invalid)	radius = -5	Error message; re-prompt	Pass
5	Dimension zero (boundary, invalid)	radius = 0	Error message; re-prompt	Pass
6	Dimension smallest valid (boundary)	radius = 0.0001	Accepted, proceeds	Pass
7	Dimension valid	radius = 5	Accepted, proceeds	Pass

5.2 Surface Area / Volume / Base Area

#	Test Case	Input	Expected Output	Actual Output
1	Sphere – Surface Area	radius = 5	314.1600 m ²	314.1600 m ² – Pass
2	Sphere – Surface Area (boundary)	radius = 2	50.2656 m ²	50.2656 m ² – Pass
3	Cylinder – Volume	radius = 2, height = 5	62.8320 m ³	62.8320 m ³ – Pass
4	Cone – Surface Area (uses iterative sqrt)	radius = 3, height = 4	75.3984 m ²	75.3984 m ² – Pass
5	Cuboid – Volume	l = 2, b = 3, h = 5	30.0000 m ³	30.0000 m ³ – Pass
6	Base Area – Sphere (disallowed)	solid = Sphere	Message; return to menu	Message shown – Pass
7	Base Area – Cone	radius = 3	28.2744 m ²	28.2744 m ² – Pass
8	Base Area – Cuboid	l = 2, b = 3	6.0000 m ²	6.0000 m ² – Pass

5.3 Mass, Weight, Potential Energy

#	Test Case	Input	Expected Output	Actual Output
1	Mass – Cuboid	$\rho = 1000$, l=2, b=3, h=5	30000.0000 kg	30000.0000 kg – Pass

#	Test Case	Input	Expected Output	Actual Output
2	Weight – Sphere	$\rho = 1000$, radius = 3	1108356.4800 N	1108356.4800 N – Pass
3	Potential Energy – Cylinder	elev.=10, $\rho=1000$, r=2, h=5	6157536.0000 J	6157536.0000 J – Pass
4	Potential Energy (boundary elevation = 0)	elev.=0, $\rho=1000$, sphere r=2	0.0000 J	0.0000 J – Pass
5	Density invalid (boundary)	$\rho = 0$	Error message; re-prompt	Pass
6	Density invalid	$\rho = -100$	Error message; re-prompt	Pass

5.4 Pressure & Buoyant Force

#	Test Case	Input	Expected Output	Actual Output
1	Pressure – Cuboid	$\rho = 1000$, l=2, b=3, h=5	49000.0000 Pa	49000.0000 Pa – Pass
2	Pressure – Sphere (disallowed)	solid = Sphere	Error; only 2–4 accepted	Pass
3	Buoyant Force – Cone	$\rho_{\text{fluid}}=1025$, r=3, h=4	378688.4640 N	378688.4640 N – Pass

5.5 Buoyancy Behavior

#	Test Case	Input	Expected Output	Actual Output
1	Solid denser than fluid	$\rho_{\text{solid}}=1200$, $\rho_{\text{fluid}}=1000$	SINKS	SINKS – Pass
2	Solid lighter than fluid	$\rho_{\text{solid}}=500$, $\rho_{\text{fluid}}=1000$	FLOATS	FLOATS – Pass
3	Equal densities (boundary)	$\rho_{\text{solid}}=1000$, $\rho_{\text{fluid}}=1000$	SUSPENDED	SUSPENDED – Pass

5.6 Menu Navigation

#	Test Case	Input	Expected Output	Actual Output
1	Invalid main-menu choice	choice = 11	Error message; menu re-displayed	Pass
2	Exit	choice = 10	Exit message; loop terminates	Pass

6. Source Code

The project was compiled with: `gcc -Wall -Wextra -std=c99 -o solids main.c geometry.c physics.c math_utils.c` — with zero warnings and zero errors.

```
solid.h
#ifndef SOLID_H
#define SOLID_H

/* ----- Symbolic constants ----- */
#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

/* ----- Solid choices ----- */
#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

/* ----- Buoyancy results ----- */
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

/* ----- geometry.c ----- */
double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

/* ----- physics.c ----- */
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);

/* ----- math_utils.c ----- */
double calculate_square_root(double number);

#endif
```

```
math_utils.c
#include <stdio.h>
#include "solid.h"

/*
 * calculate_square_root()
 * Computes the square root of 'number' iteratively using the
 * Newton-Raphson method. No function from math.h is used.
 */
double calculate_square_root(double number)
{
    double old_value, new_value, difference;

    if (number < 0)
    {
        return -1; /* real square root does not exist */
    }
}
```

```

    if (number == 0)
    {
        return 0;
    }

    old_value = number;

    do
    {
        new_value = 0.5 * (old_value + number / old_value);
        difference = new_value - old_value;

        if (difference < 0)
        {
            difference = -difference;
        }

        old_value = new_value;
    } while (difference >= TOLERANCE);

    return new_value;
}

```

geometry.c

```

#include <stdio.h>
#include "solid.h"

/* ----- Sphere ----- */
double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}

double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

/* ----- Cylinder ----- */
double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}

/* ----- Cone ----- */
double cone_area(double radius, double height)
{
    double slant_height;

    slant_height = calculate_square_root(radius * radius + height * height);

    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI * radius * radius * height;
}

/* ----- Cuboid ----- */
double cuboid_area(double length, double breadth, double height)

```

```

{
    return 2 * (length * breadth + breadth * height + length * height);
}

double cuboid_volume(double length, double breadth, double height)
{
    return length * breadth * height;
}

/* ----- Base areas (cylinder/cone share the circular base) ----- */
double circular_base_area(double radius)
{
    return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth)
{
    return length * breadth;
}

```

physics.c

```

#include <stdio.h>
#include "solid.h"

/* ----- *
 * The helper functions below take no parameters (so no pointer
 * parameters are used) and simply obtain and validate one value at
 * a time from the user. Each is local to this file.
 * ----- */

static int get_solid_choice(void)
{
    int solid_choice;

    do
    {
        printf("\nSelect the solid:\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter your choice: ");
        scanf("%d", &solid_choice);

        if (solid_choice < SPHERE || solid_choice > CUBOID)
        {
            printf("Invalid choice. Please enter a value between 1 and 4.\n");
        }
    } while (solid_choice < SPHERE || solid_choice > CUBOID);

    return solid_choice;
}

static int get_base_solid_choice(void)
{
    int solid_choice;

    do
    {
        printf("\nSelect the solid:\n");
        printf("2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter your choice: ");
        scanf("%d", &solid_choice);

        if (solid_choice < CYLINDER || solid_choice > CUBOID)
        {
            printf("Invalid choice. Please enter 2, 3, or 4.\n");
        }
    } while (solid_choice < CYLINDER || solid_choice > CUBOID);

    return solid_choice;
}

```

```
}

static double get_radius(void)
{
    double radius;

    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);

        if (radius <= 0)
        {
            printf("Invalid input. Radius must be greater than zero.\n");
        }
    } while (radius <= 0);

    return radius;
}

static double get_height(void)
{
    double height;

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);

        if (height <= 0)
        {
            printf("Invalid input. Height must be greater than zero.\n");
        }
    } while (height <= 0);

    return height;
}

static double get_length(void)
{
    double length;

    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);

        if (length <= 0)
        {
            printf("Invalid input. Length must be greater than zero.\n");
        }
    } while (length <= 0);

    return length;
}

static double get_breadth(void)
{
    double breadth;

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);

        if (breadth <= 0)
        {
            printf("Invalid input. Breadth must be greater than zero.\n");
        }
    } while (breadth <= 0);

    return breadth;
}
```

```

    }
    } while (breadth <= 0);

    return breadth;
}

static double get_density(void)
{
    double density;

    do
    {
        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);

        if (density <= 0)
        {
            printf("Invalid input. Density must be greater than zero.\n");
        }
    } while (density <= 0);

    return density;
}

/*
 * get_solid_volume()
 * Prompts for the dimensions needed for 'solid_choice' and returns the
 * corresponding volume by calling the matching geometry.c function.
 */
static double get_solid_volume(int solid_choice)
{
    double radius, height, length, breadth, volume;

    if (solid_choice == SPHERE)
    {
        radius = get_radius();
        volume = sphere_volume(radius);
    }
    else if (solid_choice == CYLINDER)
    {
        radius = get_radius();
        height = get_height();
        volume = cylinder_volume(radius, height);
    }
    else if (solid_choice == CONE)
    {
        radius = get_radius();
        height = get_height();
        volume = cone_volume(radius, height);
    }
    else
    {
        length = get_length();
        breadth = get_breadth();
        height = get_height();
        volume = cuboid_volume(length, breadth, height);
    }

    return volume;
}

/* ----- *
 * Functions declared in solid.h                *
 * ----- */

double calculate_mass(double density)
{
    int solid_choice;
    double volume;

```

```
    solid_choice = get_solid_choice();
    volume = get_solid_volume(solid_choice);

    return density * volume;
}

double calculate_weight(void)
{
    double density, mass;

    density = get_density();
    mass = calculate_mass(density);

    return mass * GRAVITY;
}

double potential_energy(double elevation)
{
    double density, mass;

    density = get_density();
    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density)
{
    int solid_choice;
    double radius, height, length, breadth;
    double volume, contact_area;

    solid_choice = get_base_solid_choice();

    if (solid_choice == CYLINDER)
    {
        radius = get_radius();
        height = get_height();
        contact_area = circular_base_area(radius);
        volume = cylinder_volume(radius, height);
    }
    else if (solid_choice == CONE)
    {
        radius = get_radius();
        height = get_height();
        contact_area = circular_base_area(radius);
        volume = cone_volume(radius, height);
    }
    else
    {
        length = get_length();
        breadth = get_breadth();
        height = get_height();
        contact_area = cuboid_base_area(length, breadth);
        volume = cuboid_volume(length, breadth, height);
    }

    return (density * volume * GRAVITY) / contact_area;
}

double buoyant_force(double fluid_density)
{
    int solid_choice;
    double volume;

    solid_choice = get_solid_choice();
    volume = get_solid_volume(solid_choice);
```

```

    return fluid_density * volume * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density)
{
    if (solid_density > fluid_density)
    {
        return SINKS;
    }
    else if (solid_density < fluid_density)
    {
        return FLOATS;
    }
    else
    {
        return SUSPENDED;
    }
}

```

main.c

```

#include <stdio.h>
#include "solid.h"

/* ----- *
 * Local input helper functions (no parameters -> no pointer
 * parameters). Each obtains and validates a single common input.
 * ----- */

static int get_solid_choice(void)
{
    int solid_choice;

    do
    {
        printf("\nSelect the solid:\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter your choice: ");
        scanf("%d", &solid_choice);

        if (solid_choice < SPHERE || solid_choice > CUBOID)
        {
            printf("Invalid choice. Please enter a value between 1 and 4.\n");
        }
    } while (solid_choice < SPHERE || solid_choice > CUBOID);

    return solid_choice;
}

static double get_radius(void)
{
    double radius;

    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);

        if (radius <= 0)
        {
            printf("Invalid input. Radius must be greater than zero.\n");
        }
    } while (radius <= 0);

    return radius;
}

static double get_height(void)

```

```
{
    double height;

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);

        if (height <= 0)
        {
            printf("Invalid input. Height must be greater than zero.\n");
        }
    } while (height <= 0);

    return height;
}

static double get_length(void)
{
    double length;

    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);

        if (length <= 0)
        {
            printf("Invalid input. Length must be greater than zero.\n");
        }
    } while (length <= 0);

    return length;
}

static double get_breadth(void)
{
    double breadth;

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);

        if (breadth <= 0)
        {
            printf("Invalid input. Breadth must be greater than zero.\n");
        }
    } while (breadth <= 0);

    return breadth;
}

static double get_density(void)
{
    double density;

    do
    {
        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);

        if (density <= 0)
        {
            printf("Invalid input. Density must be greater than zero.\n");
        }
    } while (density <= 0);
}
```

```
    return density;
}

static double get_fluid_density(void)
{
    double fluid_density;

    do
    {
        printf("Enter fluid density (kg/m^3): ");
        scanf("%lf", &fluid_density);

        if (fluid_density <= 0)
        {
            printf("Invalid input. Fluid density must be greater than zero.\n");
        }
    } while (fluid_density <= 0);

    return fluid_density;
}

static double get_solid_density(void)
{
    double solid_density;

    do
    {
        printf("Enter solid density (kg/m^3): ");
        scanf("%lf", &solid_density);

        if (solid_density <= 0)
        {
            printf("Invalid input. Solid density must be greater than zero.\n");
        }
    } while (solid_density <= 0);

    return solid_density;
}

static double get_elevation(void)
{
    double elevation;

    do
    {
        printf("Enter elevation (m): ");
        scanf("%lf", &elevation);

        if (elevation < 0)
        {
            printf("Invalid input. Elevation must not be negative.\n");
        }
    } while (elevation < 0);

    return elevation;
}

static void display_menu(void)
{
    printf("\n=====\\n");
    printf(" GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS\\n");
    printf("=====\\n");
    printf(" 1. Surface Area\\n");
    printf(" 2. Volume\\n");
    printf(" 3. Base Area\\n");
    printf(" 4. Mass\\n");
    printf(" 5. Weight\\n");
    printf(" 6. Gravitational Potential Energy\\n");
}
```

```

printf(" 7. Pressure on the Base\n");
printf(" 8. Buoyant Force\n");
printf(" 9. Buoyancy Behavior\n");
printf("10. Exit\n");
printf("-----\n");
printf("Enter your choice: ");
}

int main(void)
{
    int choice, solid_choice, buoyancy_result;
    double radius, height, length, breadth;
    double density, fluid_density, solid_density, elevation;
    double area, volume, base_area, mass, weight, energy, pressure, force;

    do
    {
        display_menu();
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: /* Surface area */
                solid_choice = get_solid_choice();

                if (solid_choice == SPHERE)
                {
                    radius = get_radius();
                    area = sphere_area(radius);
                }
                else if (solid_choice == CYLINDER)
                {
                    radius = get_radius();
                    height = get_height();
                    area = cylinder_area(radius, height);
                }
                else if (solid_choice == CONE)
                {
                    radius = get_radius();
                    height = get_height();
                    area = cone_area(radius, height);
                }
                else
                {
                    length = get_length();
                    breadth = get_breadth();
                    height = get_height();
                    area = cuboid_area(length, breadth, height);
                }

                printf("\nTotal Surface Area = %.4lf square metres\n", area);
                break;

            case 2: /* Volume */
                solid_choice = get_solid_choice();

                if (solid_choice == SPHERE)
                {
                    radius = get_radius();
                    volume = sphere_volume(radius);
                }
                else if (solid_choice == CYLINDER)
                {
                    radius = get_radius();
                    height = get_height();
                    volume = cylinder_volume(radius, height);
                }
                else if (solid_choice == CONE)
                {
                    radius = get_radius();
                    height = get_height();

```

```

        volume = cone_volume(radius, height);
    }
    else
    {
        length = get_length();
        breadth = get_breadth();
        height = get_height();
        volume = cuboid_volume(length, breadth, height);
    }

    printf("\nVolume = %.4lf cubic metres\n", volume);
    break;

case 3: /* Base area */
    solid_choice = get_solid_choice();

    if (solid_choice == SPHERE)
    {
        printf("\nBase area is not defined for a sphere.\n");
        printf("Returning to the main menu.\n");
    }
    else if (solid_choice == CYLINDER || solid_choice == CONE)
    {
        radius = get_radius();
        base_area = circular_base_area(radius);
        printf("\nBase Area = %.4lf square metres\n", base_area);
    }
    else
    {
        length = get_length();
        breadth = get_breadth();
        base_area = cuboid_base_area(length, breadth);
        printf("\nBase Area = %.4lf square metres\n", base_area);
    }
    break;

case 4: /* Mass */
    density = get_density();
    mass = calculate_mass(density);
    printf("\nMass = %.4lf kilograms\n", mass);
    break;

case 5: /* Weight */
    weight = calculate_weight();
    printf("\nWeight = %.4lf newtons\n", weight);
    break;

case 6: /* Gravitational potential energy */
    elevation = get_elevation();
    energy = potential_energy(elevation);
    printf("\nGravitational Potential Energy = %.4lf joules\n", energy);
    break;

case 7: /* Pressure on the base */
    density = get_density();
    pressure = calculate_pressure(density);
    printf("\nPressure = %.4lf pascals\n", pressure);
    break;

case 8: /* Buoyant force */
    fluid_density = get_fluid_density();
    force = buoyant_force(fluid_density);
    printf("\nBuoyant Force = %.4lf newtons\n", force);
    break;

case 9: /* Buoyancy behavior */
    solid_density = get_solid_density();
    fluid_density = get_fluid_density();
    buoyancy_result = determine_buoyancy(solid_density, fluid_density);

```

```
        if (buoyancy_result == SINKS)
        {
            printf("\nThe solid SINKS in the fluid.\n");
        }
        else if (buoyancy_result == FLOATS)
        {
            printf("\nThe solid FLOATS in the fluid.\n");
        }
        else
        {
            printf("\nThe solid remains SUSPENDED in the fluid.\n");
        }
        break;

    case 10: /* Exit */
        printf("\nExiting the program. Thank you!\n");
        break;

    default:
        printf("\nInvalid choice. Please select a value between 1 and 10.\n");
    }

} while (choice != 10);

return 0;
}
```

7. Program Execution & Verification

Below is a captured terminal transcript covering the test cases from Section 5 (compilation and multiple program runs). All actual outputs matched the expected, hand-calculated values.

Terminal Output

```
$ gcc -Wall -Wextra -std=c99 -o solids main.c geometry.c physics.c math_utils.c
(compiled with zero warnings and zero errors)
```

```
=====
RUN 1: Sphere Surface Area (valid input) -> choice 1,1,5,10
=====
```

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```
-----
Enter your choice:
```

```
Select the solid:
```

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

```
Enter your choice: Enter radius (m):
```

```
Total Surface Area = 314.1600 square metres
```

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```
-----
Enter your choice:
```

```
Exiting the program. Thank you!
```

```
=====
RUN 2: Base Area -> Cone, invalid radius (0, -5) then valid (3)
=====
```

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```

-----
Enter your choice:
Select the solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter your choice: Enter radius (m): Invalid input. Radius must be greater than zero.
Enter radius (m): Invalid input. Radius must be greater than zero.
Enter radius (m):
Base Area = 28.2744 square metres

```

```

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====

```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```

-----
Enter your choice:
Exiting the program. Thank you!

```

```

=====
RUN 3: Base Area -> Sphere (disallowed message)
=====

```

```

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====

```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```

-----
Enter your choice:
Select the solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter your choice:
Base area is not defined for a sphere.
Returning to the main menu.

```

```

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====

```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```

-----
Enter your choice:

```

Exiting the program. Thank you!

```
=====
RUN 4: Cone Surface Area (r=3,h=4) - exercises Newton-Raphson sqrt
=====
```

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice:

Select the solid:

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

Enter your choice: Enter radius (m): Enter height (m):

Total Surface Area = 75.3984 square metres

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice:

Exiting the program. Thank you!

```
=====
RUN 5: Mass -> Cuboid (rho=1000, l=2,b=3,h=5)
=====
```

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice: Enter density (kg/m³):

Select the solid:

1. Sphere
2. Cylinder
3. Cone
4. Cuboid

Enter your choice: Enter length (m): Enter breadth (m): Enter height (m):

Mass = 30000.0000 kilograms

```

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
-----
Enter your choice:
Exiting the program. Thank you!

=====
RUN 6: Weight -> Sphere (rho=1000, r=3)
=====

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
-----
Enter your choice: Enter density (kg/m^3):
Select the solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter your choice: Enter radius (m):
Weight = 1108356.4800 newtons

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
-----
Enter your choice:
Exiting the program. Thank you!

=====
RUN 7: Potential Energy -> Cylinder (elev=10, rho=1000, r=2,h=5)
=====

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
1. Surface Area
2. Volume

```

```
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
```

```
-----
Enter your choice: Enter elevation (m): Enter density (kg/m^3):
Select the solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter your choice: Enter radius (m): Enter height (m):
Gravitational Potential Energy = 6157536.0000 joules
```

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

```
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
```

```
-----
Enter your choice:
Exiting the program. Thank you!
```

```
=====
RUN 8: Potential Energy boundary -> elevation = 0
=====
```

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

```
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
```

```
-----
Enter your choice: Enter elevation (m): Enter density (kg/m^3):
Select the solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter your choice: Enter radius (m):
Gravitational Potential Energy = 0.0000 joules
```

```
=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
```

```
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
```

```

8. Buoyant Force
9. Buoyancy Behavior
10. Exit
-----
Enter your choice:
Exiting the program. Thank you!

=====
RUN 9: Pressure -> Cuboid (rho=1000, l=2,b=3,h=5)
=====

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
-----
Enter your choice: Enter density (kg/m^3):
Select the solid:
2. Cylinder
3. Cone
4. Cuboid
Enter your choice: Enter length (m): Enter breadth (m): Enter height (m):
Pressure = 49000.0000 pascals

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
-----
Enter your choice:
Exiting the program. Thank you!

=====
RUN 10: Buoyant Force -> Cone (rho_fluid=1025, r=3,h=4)
=====

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====
1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit
-----
Enter your choice: Enter fluid density (kg/m^3):
Select the solid:
1. Sphere
2. Cylinder

```

```

3. Cone
4. Cuboid
Enter your choice: Enter radius (m): Enter height (m):
Buoyant Force = 378688.4640 newtons

```

```

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====

```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```

-----
Enter your choice:
Exiting the program. Thank you!

```

```

=====
RUN 11: Buoyancy Behavior -> Sinks / Floats / Suspended
=====

```

```

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====

```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```

-----
Enter your choice: Enter solid density (kg/m^3): Enter fluid density (kg/m^3):
The solid SINKS in the fluid.

```

```

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====

```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

```

-----
Enter your choice: Enter solid density (kg/m^3): Enter fluid density (kg/m^3):
The solid FLOATS in the fluid.

```

```

=====
GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS
=====

```

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior

10. Exit

Enter your choice: Enter solid density (kg/m³): Enter fluid density (kg/m³):
The solid remains SUSPENDED in the fluid.

=====

GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS

=====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice:
Exiting the program. Thank you!

=====

RUN 12: Invalid main-menu choice (11) then Exit

=====

=====

GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS

=====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice:
Invalid choice. Please select a value between 1 and 10.

=====

GEOMETRICAL & PHYSICAL PROPERTIES OF SOLIDS

=====

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on the Base
8. Buoyant Force
9. Buoyancy Behavior
10. Exit

Enter your choice:
Exiting the program. Thank you!

8. Conclusion

A menu-driven, modular C project was successfully designed, developed, compiled, and tested to compute the geometrical and physical properties of a sphere, cylinder, cone, and cuboid. The project was organised into `solid.h`, `math_utils.c`, `geometry.c`, `physics.c`, and `main.c` as specified. All functions use call by value with fundamental data types only; the square root required for the cone's slant height was implemented iteratively using the Newton–Raphson method instead of using `sqrt()` from `math.h`. Every input is validated immediately at the point it is obtained, using do-while loops that re-prompt on invalid entries. The program was tested against valid, invalid, and boundary test cases for every operation, and all actual outputs matched the expected results, confirming that the implementation satisfies the given requirements.